

MC521 - Relatório II

Júlio da Silva Telles

17 de Junho de 2026

1 C - Orac and Models (CodeForces - 1350B)

O problema consiste em encontrar o tamanho máximo de uma sequência de índices $i_1 < i_2 < \dots < i_k$, tal que cada índice na sequência seja um múltiplo do índice anterior (ou seja, i_x divide i_{x+1}) e os valores correspondentes no vetor original sejam estritamente crescentes ($v[i_x] < v[i_{x+1}]$).

1.1 Solução

Podemos resolver este problema utilizando Programação Dinâmica (DP) de forma iterativa, com uma complexidade de tempo de $\mathcal{O}(N \log N)$ graças à soma da série harmônica ao iterar pelos múltiplos. O algoritmo segue os seguintes passos:

1. **Inicialização do Estado:** Inicializamos um vetor de estados da DP ($\mathbf{s}$) com o valor base 1 para todos os elementos a partir da metade do array, pois o tamanho mínimo de qualquer sequência válida é o próprio elemento sozinho.
2. **Iteração e Transição (Bottom-Up):** Percorremos os índices do array de trás para frente, de $N/2$ até 1. Para cada índice, verificamos todos os seus múltiplos possíveis (multiplicando o índice por $j = 2, 3, \dots$).
3. **Validação e Atualização:** Para cada múltiplo válido, verificamos se o valor no array original satisfaz a condição de crescimento estrito. Se for menor, testamos se incluir este múltiplo melhora a nossa sequência local ($\mathbf{s}[i] < 1 + \mathbf{s}[i * j]$). Se melhorar, atualizamos o valor de $\mathbf{s}[i]$ e guardamos o tamanho máximo global encontrado para impressão no final de cada caso de teste.

2 G - Partial Teacher (CodeForces-67A)

O problema exige a atribuição de valores a um conjunto de V vértices de forma a respeitar condições de desigualdade direcionais ("L" para menor, "R" para maior) ou igualdade ("=") fornecidas em formato de string. O objetivo é calcular o valor acumulado ou o tamanho do maior caminho direcionado para cada nó, respeitando essas relações.

2.1 Solução

Podemos modelar o problema como a busca do caminho mais longo em um Grafo Direcionado Acíclico (DAG) utilizando Busca em Profundidade (DFS) com *Memoization* (Programação Dinâmica em grafos). O algoritmo segue os seguintes passos:

1. **Construção do Grafo:** Lemos a string de relações e interpretamos os caracteres como arestas. Se a relação for "L", criamos uma aresta do nó atual para o próximo (indicando precedência). Se for "R", a direção da aresta é invertida. Se for "=", armazenamos essa equivalência em um vetor auxiliar (`equals`) em vez de criar uma aresta.
2. **Busca em Profundidade (Memoization):** Para cada vértice do grafo, iniciamos uma travessia DFS (`dpfs`). Durante a recursão, visitamos todos os nós adjacentes e calculamos o tamanho do maior caminho encontrado. O resultado é armazenado no vetor `sol` para evitar que subproblemas sejam recalculados desnecessariamente.
3. **Tratamento de Igualdades e Retorno:** A base da recursão inclui a verificação do vetor `sol`. Ao terminarmos de calcular o valor de um nó, checamos se ele possui um nó adjacente marcado como igual. Se sim, propagamos o mesmo valor máximo de caminho para o nó equivalente. Por fim, imprimimos o valor processado para todos os vértices.